



**Faculty of Computer Applications &
Information Technology**

iMScIT Programme- Sem-6

SOFTWARE TESTING
221604603

Faculty of Computer Applications & IT

Software Testing -Introduction

- Software testing is an important process in the software development lifecycle .
- It involves verifying and validating that a software application is free of bugs, meets the technical requirements set by its design and development , and satisfies user requirements efficiently and effectively.
- This process ensures that the application can handle all exceptional and boundary cases, providing a robust and reliable user experience. By systematically identifying and fixing issues, software testing helps deliver high-quality software that performs as expected in various scenarios.
- We can say that in the era of software testing, the terms bugs, defects, error, fault, and failure come across every second of the day.

Software Testing

- Software Testing is a method to assess the functionality of the software program.
- Software Testing is the process checks whether the actual software matches the expected requirements and ensures the software is bug-free.
- The purpose of software testing is to identify the errors, faults, or missing requirements in contrast to actual requirements.
- It mainly aims at measuring the specification, functionality, and performance of a software program or application.

Necessity Of Testing

- **Defects can be identified early:** Software testing is important because if there are any bugs they can be identified early and can be fixed before the delivery of the software.
- **Improves quality of software:** Software Testing uncovers the defects in the software, and fixing them improves the quality of the software.
- **Increased customer satisfaction:** Software testing ensures reliability, security, and high performance which results in saving time, costs, and customer satisfaction.

Necessity Of Testing

- **Helps with scalability:** Software testing type non-functional testing helps to identify the scalability issues and the point where an application might stop working.
- **Saves time and money:** After the application is launched it will be very difficult to trace and resolve the issues, as performing this activity will incur more costs and time. Thus, it is better to conduct software testing at regular intervals during software development.

Necessity Of Testing

- **Low Failure** : Failure of an application impacts its working and brand value. Software testing helps in knowing the cases where a particular application is most likely to fail.
- For example, An eCommerce application is more likely to fail when there is heavy traffic and load. This type of application goes through stress testing to check the highest possible limit of the application to handle the load. Software testing makes an application more robust and reliable.

Necessity Of Testing

- **Security:** The main concern of the digital world is Security. A secure system always remains on the priority list of customers. Owners pay multiple dollars to make their systems secure from hackers, malicious attacks, and other thefts.
- Security testing technique is used to identify the security level of the application and testers try to find loopholes to break the security of an application.
- In the case of bank applications, security is the foremost requirement as it deals with customer money, and bank applications always remain on top of the thefts.
- The testing team uses security testing to identify defects and the development team tries to cover the application with multiple security layers.
- Thus, software testing is a must to maintain the security of an application.

Necessity Of Testing

- Software bugs can cause potential monetary and human loss.
- There are many examples in history that clearly depicts that without the testing phase in software development lot of damage was incurred. Below are some examples:
- **1985:** Canada's Therac-25 radiation therapy malfunctioned due to a software bug and resulted in lethal radiation doses to patients leaving 3 injured and 3 people dead.
- **1994:** China Airlines Airbus A300 crashed due to a software bug killing 264 people.
- **1996:** A software bug caused U.S. bank accounts of 823 customers to be credited with 920 million US dollars.
- **1999:** A software bug caused the failure of a \$1.2 billion military satellite launch.

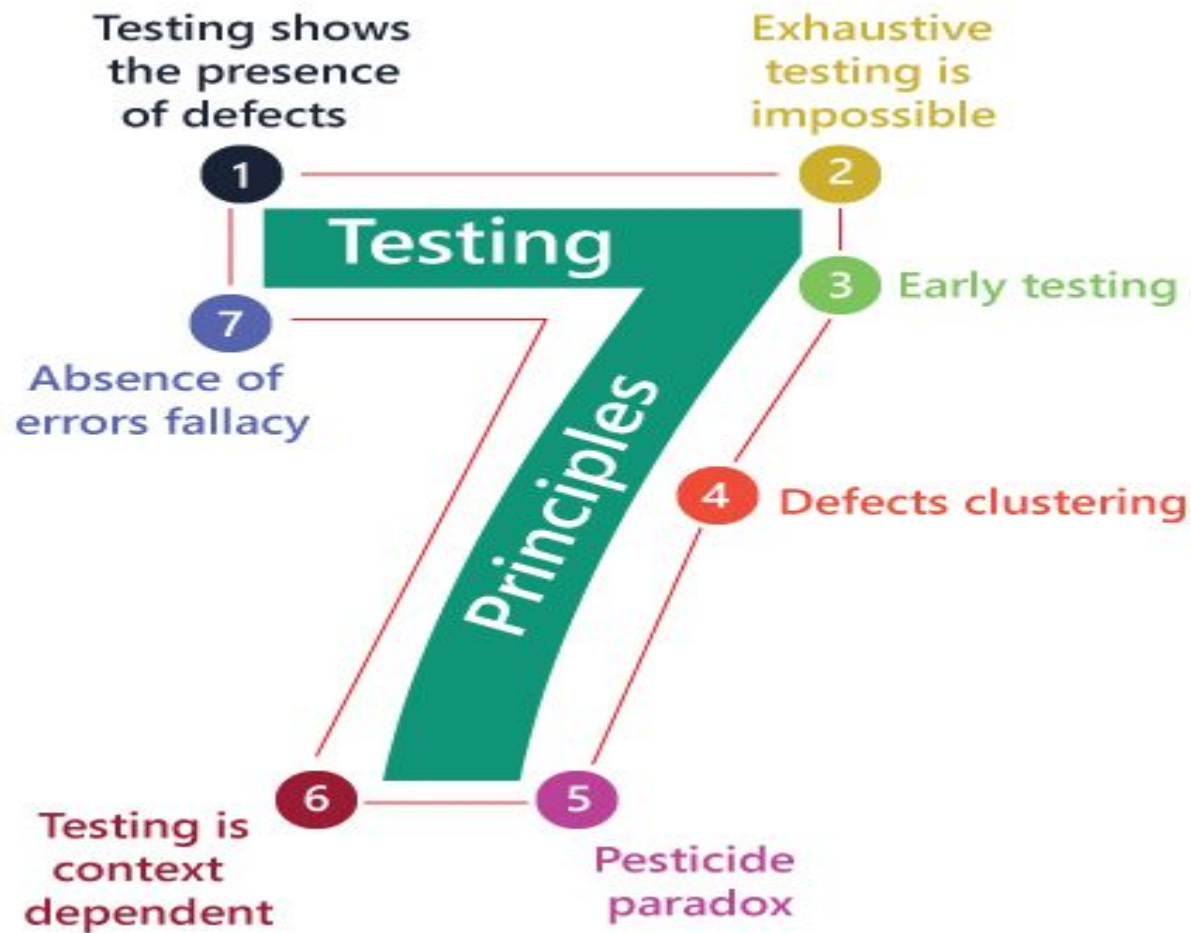
Necessity Of Testing

- **2015:** A software bug in fighter plan F-35 resulted in making it unable to detect targets correctly.
- **2015:** Bloomberg terminal in London crashed due to a software bug affecting 300,000 traders on the financial market and forcing the government to postpone the 3bn pound debt sale.
- **Starbucks** was forced to close more than 60% of its outlet in the U.S. and Canada due to a software failure in its POS system.
- **Nissan cars** were forced to recall 1 million cars from the market due to a software failure in the car's airbag sensory detectors.

Testing Principles

- **Testing shows the presence of defects**
- **Exhaustive testing is not possible**
- **Early testing**
- **Defect clustering**
- **Pesticide paradox**
- **Testing is Context-Dependent**
- **Absence of Errors fallacy**

Testing Principles



Testing Principles – 1) Testing shows the presence of defects

- The goal of software testing is to make the software fail. Software testing reduces the presence of defects.
- The test engineer will test the application to make sure that the application is bug or defects free.
- The primary purpose of doing testing is to identify the numbers of unknown bugs with the help of various methods and testing techniques to find any defects that might cause the product failure to meet the client's needs.

Testing Principles – 1) Testing shows the presence of defects

- By doing testing on any application, we can decrease the number of bugs, which does not mean that the application is defect-free because sometimes the software seems to be bug-free while performing multiple types of testing on it.
- But at the time of deployment in the production server, if the end-user encounters those bugs which are not found in the testing process.

Testing Principles – 2) Exhaustive Testing Is Not Possible

- It is the process of testing the functionality of the software in all possible inputs (valid or invalid) and pre-conditions is known as **exhaustive testing**.
- Exhaustive testing is impossible means the software can never test at every test case.
- It can test only some test cases and assume that the software is correct and it will produce the correct output in every test case.
- If the software will test every test case then it will take more cost, effort, etc., which is impractical.

Testing Principles – 2) Exhaustive Testing Is Not Possible

- Yes! Exhaustive testing is not possible. Instead, we need the optimal amount of testing based on the risk assessment of the application.
- And the million dollar question is, how do you determine this risk?
- To answer this let's do an exercise
- In your opinion, Which operation is most likely to cause your Operating system to fail?
- I am sure most of you would have guessed, Opening 10 different application all at the same time.
- So if you were testing this Operating system, you would realize that defects are likely to be found in multi-tasking activity and need to be tested thoroughly

Testing Principles – 3) Early Testing

- To find the defect in the software, early test activity shall be started. The defect detected in the early phases of SDLC will be very less expensive.
- For better performance of software, software testing will start at the initial phase i.e. testing will perform at the requirement analysis phase.
- To perform testing, we will require the requirement specification documents; therefore, if the requirements are defined incorrectly, then it can be fixed directly rather than fixing them in another stage, which could be the development phase.

Testing Principles – 4) Defect Clustering

- In a project, a small number of modules can contain most of the defects.
- The Pareto Principle (80/20 rule) for software testing states that 80% of software defects come from 20% of modules.

Testing Principles – 5) Pesticide Paradox

- Repetitive use of the same pesticide mix to eradicate insects during farming will over time lead to the insects developing resistance to the pesticide. Thereby ineffective of pesticides on insects.
- The same applies to software testing. **If the same set of repetitive tests are conducted, the method will be useless for discovering new defects.**
- To overcome this, the test cases need to be regularly reviewed & revised, adding new & different test cases to help find more defects.

Testing Principles – 6) Testing Is Context Dependent

- Testing is context dependent which basically means that the way you test an e-commerce site will be different from the way you test a commercial off the shelf application.
- All the developed software's are not identical. You might use a different approach, methodologies, techniques, and types of testing depending upon the application type.
- For instance testing, any POS system at a retail store will be different than testing an ATM machine.

Testing Principles – 7) Absence Of Errors -Fallacy

- It is possible that software which is 99% bug-free is still unusable. This can be the case if the system is tested thoroughly for the wrong requirement.
- Software testing is not mere finding defects, but also to check that software addresses the business needs.
- The absence of Error is a Fallacy i.e. Finding and fixing defects does not help if the system build is unusable and does not fulfill the user's needs & requirements.

Verification And Validation

- Verification and Validation is the process of investigating whether a software system satisfies specifications and standards and fulfills the required purpose.
- Verification and Validation both play an important role in developing good software development.
- **Verification** helps in examining whether the product is built right according to requirements, while **validation** helps in examining whether the right product is built to meet user needs.
- **Verification is a static process.** It involves reviewing and analyzing documentation and design without actually executing any code. On the other hand, **validation is a dynamic process.** It involves executing the software system to ensure functionality, usability, and suitability.

What is Verification?

- Verification is the process of checking that software achieves its goal without any bugs.
- It is the process to ensure whether the product that is developed is right or not.
- It verifies whether the developed product fulfills the requirements that we have.
- **Verification is static testing.**
- **Verification means Are we building the product right?**

What is Verification?

- **Verification Example:**
- Imagine a team is developing a new mobile banking app. During the verification phase, they review the requirements and design documents.
- They check if all the specified features like fund transfer, account balance check, and transaction history are included and correctly detailed in the design.
- They also perform peer reviews and inspections to ensure the design aligns with the requirements.
- This step ensures that the app is being built according to the initial plan and specifications without actually running the app.

What is Validation?

- Validation is the process of checking whether the software product is up to the mark or in other words product has high-level requirements.
- It is the process of checking the validation of the product i.e. it checks what we are developing is the right product.
- It is validation of the actual and expected products.
- **Validation is dynamic testing.**
- **Validation means Are we building the right product?**

What is Validation?

- **Validation Example**
- In the validation phase, the team starts testing the mobile banking app on actual devices.
- They check if users can log in, transfer money, and view their transaction history as intended.
- Testers perform usability tests to ensure the app is user-friendly and functional tests to ensure all features work correctly.
- They might also involve real users to provide feedback on the app's performance.
- This phase ensures that the app works as expected and meets user needs in real-world scenarios.

Verification V/S Validation

	Verification	Validation
Definition	Verification refers to the set of activities that ensure software correctly implements the specific function	Validation refers to the set of activities that ensure that the software that has been built is traceable to customer requirements.
Focus	It includes checking documents, designs, codes, and programs.	It includes testing and validating the actual product.
Type of Testing	Verification is the <u>static testing</u> .	Validation is <u>dynamic testing</u> .
Execution	It does <i>not</i> include the execution of the code.	It includes the execution of the code.
Methods Used	Methods used in verification are reviews, <u>walkthroughs</u> , inspections and desk-checking.	Methods used in validation are <u>Black Box Testing</u> , <u>White Box Testing</u> and <u>non-functional testing</u> .
Purpose	It checks whether the software conforms to specifications or not.	It checks whether the software meets the requirements and expectations of a customer or not.
Bug	It can find the <u>bugs</u> in the early stage of the development.	It can only find the bugs that could not be found by the verification process.
Goal	The goal of verification is application and software architecture and specification.	The goal of validation is an actual product.

Verification V/S Validation

	Verification	Validation
Responsibility	Quality assurance team does verification.	Validation is executed on software code with the help of testing team.
Timing	It comes before validation.	It comes after verification.
Human or Computer	It consists of checking of documents/files and is performed by human.	It consists of execution of program and is performed by computer.
Lifecycle	After a valid and complete specification the verification starts.	Validation begins as soon as project starts.
Error Focus	Verification is for prevention of errors.	Validation is for detection of errors.

STLC – Software Testing Life Cycle

- STLC stands for Software Testing Life Cycle.
- STLC is a process where all the components and features of an application are tested completely such that they satisfy the requirements and quality as per the standard.
- STLC involves strategic testing that needs to be carried out in each phase of development.
- STLC is a fundamental part of the Software Development Life Cycle (SDLC) but STLC consists of only the testing phases.
- STLC starts as soon as requirements are defined or software requirement document is shared by stakeholders.
- STLC yields a step-by-step process to ensure quality software.

STLC – Software Testing Life Cycle

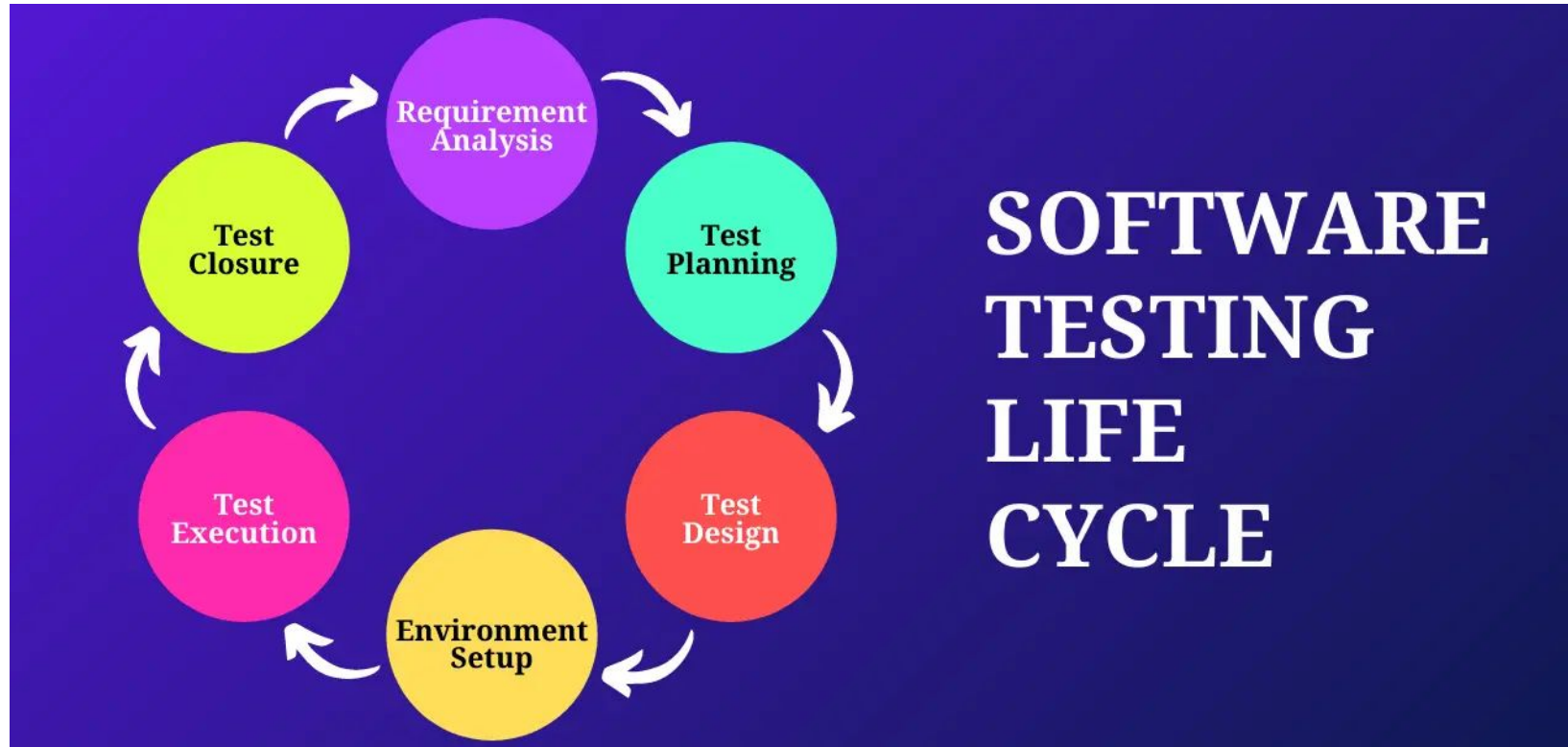
- “Prevention is better than cure”, As we are all aware, preventing a problem from occurring is better than investing a huge amount of money, resources, and time to resolve a problem that has occurred.
- This is the sole reason for the Software Testing Life Cycle, which denotes that testing needs to be done in each phase of the development process to ensure a top-quality product within the scheduled time, cost, and resources.
- By incorporating the STLC process, testing will be done at each level of development for a component.

STLC – Software Testing Life Cycle

- Software testing life cycle contains the following steps:

- ☐ **Requirement Analysis**
- ☐ **Test Plan Creation**
- ☐ **Test Case Development**
- ☐ **Environment setup**
- ☐ **Test case Execution & Defect Logging**
- ☐ **Test Cycle Closure**

STLC – Software Testing Life Cycle



STLC – 1) Requirement Analysis

- Requirement Analysis is the first step of the Software Testing Life Cycle (STLC).
- In this phase quality assurance team understands the requirements like what is to be tested.
- If anything is missing or not understandable then the quality assurance team meets with the stakeholders to better understand the detailed knowledge of requirements.
- At the end of this stage, the testing team should have a clear understanding of the software requirements and should have identified any potential issues that may impact the testing process.

STLC – 1) Requirement Analysis

- **The activities that take place during the Requirement Analysis stage include:**
- Reviewing the software requirements document (SRD) and other related documents
- Interviewing stakeholders to gather additional information
- Identifying any ambiguities or inconsistencies in the requirements
- Identifying any missing or incomplete requirements
- Identifying any potential risks or issues that may impact the testing process

STLC – 1) Requirement Analysis

- **Entry Criteria:**
 - Business Requirement Document, acceptance criteria
 - Reviewing the software requirements document (SRD)
 - Based on the requirements, the types of testing that are needed will be determined.
- **Exit Criteria:**
 - Requirement traceability matrix (RTM) – It is a document that maps and traces user requirements with test cases that reflects the coverage.
 - Automation Feasibility Document(If required)

STLC – 2) Test Planning

- Test Planning is the most efficient phase of the software testing life cycle where all testing plans are defined.
- In this phase manager of the testing, team calculates the estimated effort and cost for the testing work.
- This phase gets started once the requirement-gathering phase is completed.
- The quality assurance professional will prepare a test plan. As “what needed to be tested” is determined, the QA Lead /QA Team will prepare estimates for work (time) and determine a test strategy.
- Basically, it's a phase where we determine the types of testing needed, test tool selection, time and / Effort required, and resource requirements.

STLC – 2) Test Planning

- **The activities that take place during the Test Planning stage include:**
- Identifying the testing objectives and scope
- Developing a test strategy: selecting the testing methods and techniques that will be used
- Identifying the testing environment and resources needed
- Identifying the test cases that will be executed and the test data that will be used
- Estimating the time and cost required for testing
- Identifying the test deliverables and milestones
- Assigning roles and responsibilities to the testing team
- Reviewing and approving the test plan

STLC – 2) Test Planning

- **Entry Criteria :**
- Requirement Documents
- Requirement Traceability Matrix Document
- **Exit Criteria:**
- Approved Test Plan document, Testing Techniques, Timeline, Cost, Risks, and Resource allocation.

STLC – 3) Test Case Development

- The test case development phase gets started once the test planning phase is completed.
- In this phase testing team notes down the detailed test cases.
- The testing team also prepares the required test data for the testing.
- When the test cases are prepared then they are reviewed by the quality assurance team.
- At the end of this stage, the testing team should have a set of comprehensive and accurate test cases that provide adequate coverage of the software or application.

STLC – 3) Test Case Development

- **The activities that take place during the Test Case Development stage include:**
- Identifying the test cases that will be developed
- Writing test cases that are clear, concise, and easy to understand
- Creating test data and test scenarios that will be used in the test cases
- Identifying the expected results for each test case
- Reviewing and validating the test cases
- Updating the requirement traceability matrix (RTM) to map requirements to test cases

STLC – 3) Test Case Development

- **Entry Criteria :**
- Approved Test Plan document, Timeline, Cost, Risks, and Resource allocation
- Requirement Traceability Matrix
- **Exit Criteria:**
- Test Case Documentation/Test Script
- Identified Test Data

STLC – 4) Environment Setup

- Test environment setup is a vital part of the STLC.
- Basically, the test environment decides the conditions on which software is tested. Test Environment Setup determines the software and hardware conditions under which the system is tested.
- **This is independent activity and can be started along with test case development.**
- **In this process, the testing team is not involved. either the developer or the customer creates the testing environment.**
- However, QA performs **a smoke test (i.e., High-level testing to confirm that the environment is blocker free and is ready for testing)** to check the readiness of the environment for testing.

STLC – 4) Environment Setup

- **Entry Criteria :**
- Test strategy and test plan document.
- Test case document.
- Testing data.
- Environment set-up plan (if any)
- **Exit Criteria:**
- Environment ready with test data set up
- Smoke Test Report

STLC – 5) Test Execution

- Test execution is the actual phase where the features are tested.
- Testers start executing the test cases/Test script against the build to validate its behavior.
- Test case executions are done either manually or using automation tools.
- The defects found on testing the build are logged as bugs and reported to the developer and a bug report is submitted on the same.
- Based on it Test cases /Test procedures status are updated (e.g., Status – New, Pass, Fail, Rejected, blocked, closed, etc.)

STLC – 5) Test Execution

- **The activities that take place during the test execution stage of the Software Testing Life Cycle (STLC) include:**
- Test execution: The test cases and scripts created in the test design stage are run against the software application to identify any defects or issues.
- Defect logging: Any defects or issues that are found during test execution are logged in a defect tracking system, along with details such as the severity, priority, and description of the issue.
- Test data preparation: Test data is prepared and loaded into the system for test execution
- Test environment setup: The necessary hardware, software, and network configurations are set up for test execution

STLC – 5) Test Execution

- **The activities that take place during the test execution stage of the Software Testing Life Cycle (STLC) include:**
- Test execution: The test cases and scripts are run, and the results are collected and analyzed.
- Test result analysis: The results of the test execution are analyzed to determine the software's performance and identify any defects or issues.
- Defect retesting: Any defects that are identified during test execution are retested to ensure that they have been fixed correctly.
- Test Reporting: Test results are documented and reported to the relevant stakeholders.

STLC – 5) Test Execution

- **Entry Criteria:**

- Test strategy and test plan document.
- Test case document.
- Testing data.
- Testing Environment
- Testing Tool(if any)

- **Exit Criteria:**

- Test Case -Execution Result
- RTM with execution status
- Defect Report

STLC – 6) Test Closure

- This is the final phase which delivers a complete test result report.
- It involves a summary of the entire Testing process – Objectives met, Test Case Execution Report, Test coverage, Defect Report, Time taken, Cost, etc.
- The main objective of the test closure stage is to ensure that all testing-related activities have been completed and that the software is ready for release.
- At the end of the test closure stage, the testing team should have a clear understanding of the software's quality and reliability, and any defects or issues that were identified during testing should have been resolved. The test closure stage also includes documenting the testing process and any lessons learned so that they can be used to improve future testing processes

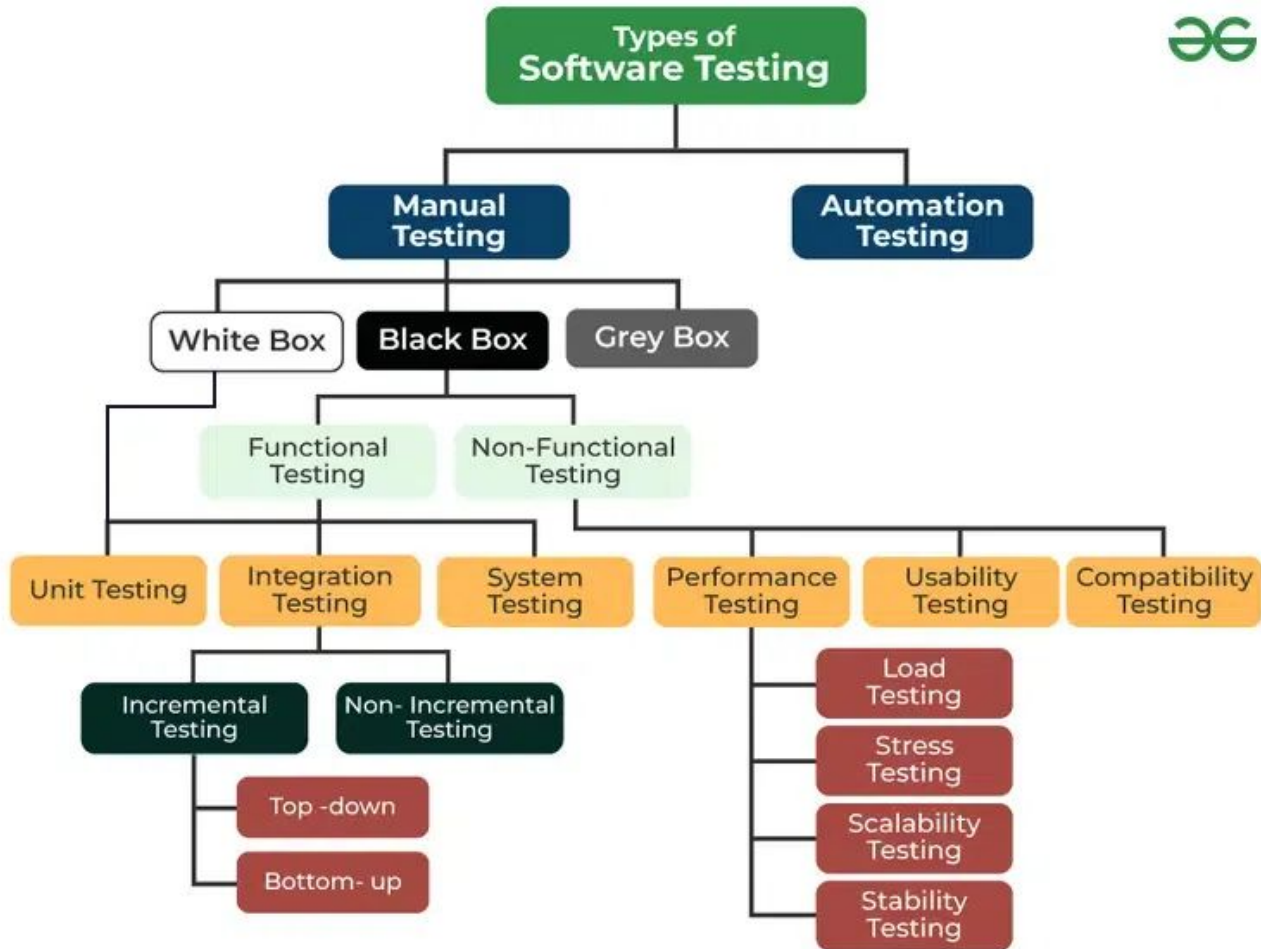
STLC – 6) Test Closure

- **Entry Criteria:**
- Test Execution Report, Defect Report
- **Exit Criteria:**
- Test Closure Report
- Summary Report

Types Of Testing

- There are two different types of software testing currently used in the industry both have their own advantages and disadvantages.
- Different Types of Software Testing :
- **Manual Testing**
- **Automation Testing**

Types Of Testing



Types Of Testing – Automation testing

- **Automated Testing is a technique where the Tester writes scripts on their own and uses suitable Software or Automation Tool to test the software.**
- It is an Automation Process of a Manual Process.
- It allows for executing repetitive tasks without the intervention of a Manual Tester.
- Automation testing is the best way to enhance the efficiency, productivity, and coverage of Software testing.

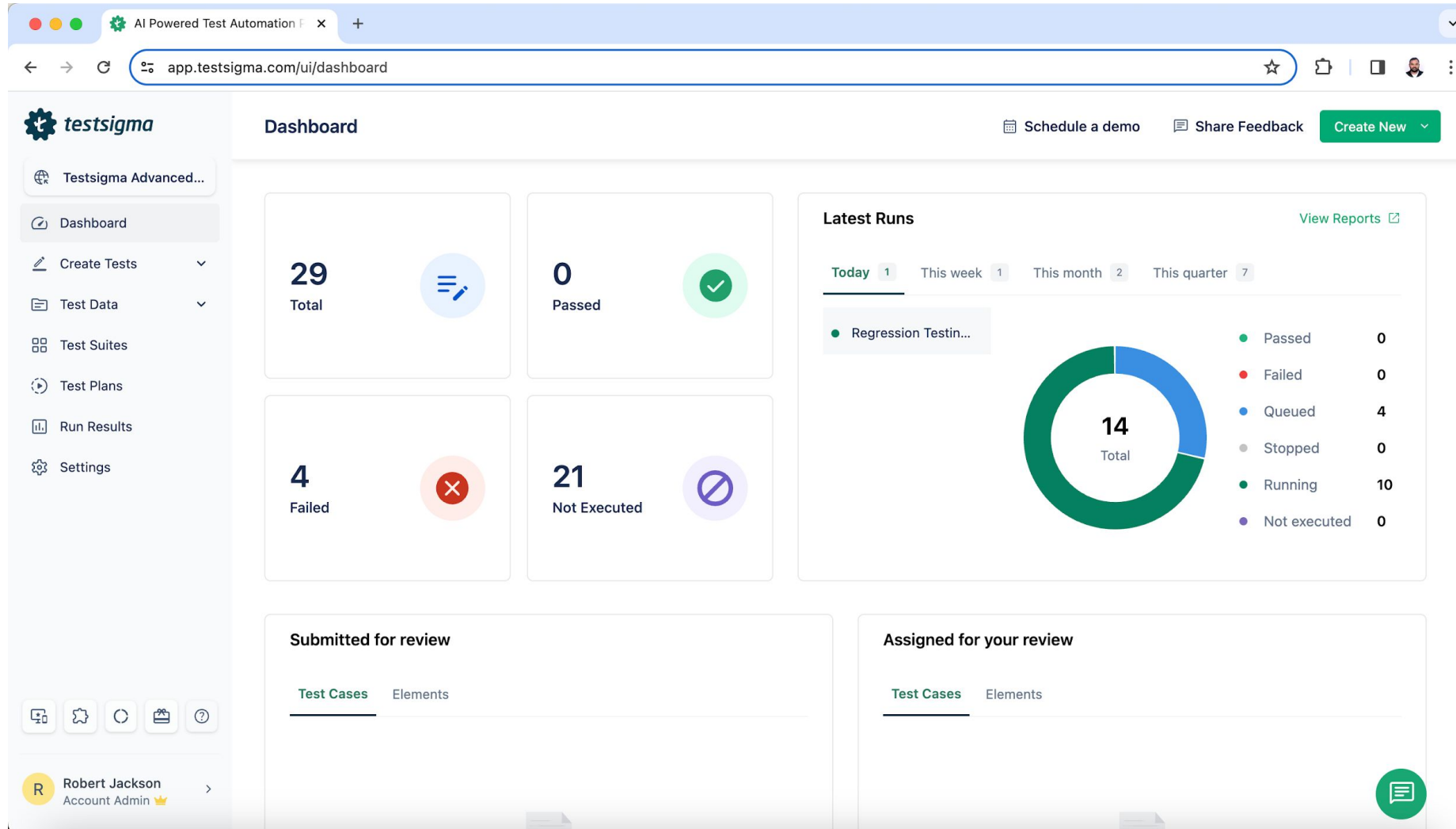
Automation testing – Introduction To Automated Testing Framework

- A testing framework is a set of guidelines or rules used for creating and designing test cases.
- A framework is comprised of a combination of practices and tools that are designed to help QA professionals test more efficiently.
- In short, a test automation framework is a set of components that facilitate executing tests and comprehensive reporting of test results.
- These frameworks are like blueprints for making automated tests. They help you create efficient and repeatable tests that can be used on different platforms and applications.

Automation testing – Introduction To Automated Testing Framework

- The most popular and easy-to-use automation frameworks include **Testsigma, Selenium, Appium, Cucumber, and Robot Framework.**
- Each framework has its strengths and weaknesses, and which one you choose depends on your specific needs and requirements.
- **Testsigma** allows you to create automated test cases in simple English scripts, enabling everyone to create and run testcases.

Types Of Testing – Automation testing



Automation testing – Cucumber Framework

- Cucumber enables developers to create automated tests efficiently through user stories.
- These are written in plain language that can be mapped to computerized tests.
- **Cucumber's easy syntax is expressed using plain language text called Gherkin**, which is written in a way that makes it easy for non-programmers to understand the behavior of an application.
- An example of Gherkin syntax is: Given I am on the homepage When I click the "Sign Up" button Then I should be taken to the signup page.

Automation testing – Cucumber Framework

- **Cross-platform:** Cucumber is open for multiple platforms, including Windows, Mac, and Linux.
- **Support for multiple languages:** Cucumber supports various programming languages, such as Java, Ruby, and Python.
- **Reporting:** Cucumber offers vast reporting of test results, enabling the identification of areas requiring improvement.

Automation testing – PyTest Framework

- Pytest is one of the best open-source, simple, scalable and Python-based Test Automation Framework available at the market today.
- Pytest powers you to test anything including Databases, UI and is more popular among testers for API Testing.
- It can either be run as an individual tool for testing or it can be attached to Python web development frameworks like Django or Flask, enhancing their unit test capabilities and reliability.
- Tests in Pytest are basically python functions and we can run tests as whole, or specifically based on tags or names.

Types Of Testing – Manual testing

- **Testing any software or an application according to the client's needs without using any automation tool is known as manual testing.**
- In manual software testing, a tester carries out tests on the software by following a set of predefined test cases.
- Manual testing is done in person, by clicking through the application or interacting with the software and APIs with the appropriate tooling.
- In this testing, testers make test cases for the codes, test the software, and give the final report about that software.
- Manual testing is time-consuming because it is done by humans, and there is a chance of human errors.

Types Of Testing – Manual testing

- **Types of Manual Testing:**
- White Box Testing
- Black Box Testing
- Gray Box Testing

Manual testing – White Box Testing

- **White box testing is a software testing technique that involves testing the internal structure and workings of a software application.**
- The tester has access to the source code and uses this knowledge to design test cases that can verify the correctness of the software at the code level.
- White box testing techniques analyze the internal structures the used data structures, internal design, code structure, and the working of the software rather than just the functionality as in black box testing.
- **It is also called glass box testing clear box testing or structural testing.**
- **White Box Testing is also known as transparent testing or open box testing.**

Manual testing – Black Box Testing

- **Black-box testing is a type of software testing in which the tester is not concerned with the internal knowledge or implementation details of the software but rather focuses on validating the functionality based on the provided specifications or requirements.**
- **Advantages of Black Box Testing:**
- The tester does not need to have more functional knowledge or programming skills to implement the Black Box Testing.
- It is efficient for implementing the tests in the larger system.
- Tests are executed from the user's or client's point of view.
- Test cases are easily reproducible.
- It is used to find the ambiguity and contradictions in the functional specifications.

Manual testing – Grey Box Testing

- **Gray Box Testing is a software testing technique that is a combination of the Black Box Testing technique and the White Box Testing technique.**
- In the Black Box Testing technique, the tester is unaware of the internal structure of the item being tested and in White Box Testing the internal structure is known to the tester.
- **The internal structure is partially known in Gray Box Testing.**
- This includes access to internal data structures and algorithms to design the test cases.

Manual testing V/s Automation Testing

Manual Testing	Automation Testing
In manual testing, the test cases are executed by the human tester.	In automated testing, the test cases are executed by the software tools.
Manual testing is time-consuming.	Automation testing is faster than manual testing.
Manual testing takes up human resources.	Automation testing takes up automation tools and trained employees.
Exploratory testing is possible in manual testing.	Exploratory testing is not possible in automation testing.
Manual testing doesn't use frameworks.	Automation testing uses frameworks like Data Drive, Keyword, etc.